Singapore Elections Live Map

Singapore Elections Live Map *(student proposal)*

> **Assessment:** ◆ Fit with adjustments · **Category:** GovTech · **Assessed:** 2026-07-05 · **Status:** awaiting instructor approval

Proposal (as submitted): A world representing the Singapore map, showing an election preview for the various districts. Real-time feeds during election day: live vote counts, live results, and comparison of the various political parties' results.

Adjustments required for approval:

- A **C++ election-feed simulator must drive the system** (GovTech pool M1 G14): real election feeds won't be live during the trimester, so seeded, configurable vote-arrival scenarios (turnout curves, batch arrivals per district) are what make the real-time loop load-bearing and every demo reproducible.
- The map must be **custom-rendered 2D** — district polygons (e.g. from data.gov.sg GeoJSON boundaries) tessellated and drawn with the team's own OpenGL renderer, with choropleth coloring and animated transitions. Embedding a web map library would void the 2D requirement.
- **Tallies and projections computed in the C++ service** — running counts, swing vs. baseline, winner projections — with Firebase used only for persistence and fan-out to viewers, not as the computation engine.

References:

- Elections Department Singapore — official, incl. electoral division maps
- 270toWin — interactive election results maps
- data.gov.sg — Singapore open data portal (election datasets, GeoJSON boundaries)

Core Tech Requirements

Legend: ✔ Applies directly | ◆ Applies with domain adaptation | ● Optional | ✘ Not needed | » Second trimester

Core Tech Requirement	Singapore Elections Live Map
External Database (Firebase)	✔
Authentication »	✔
Analytics »	✔
C++ Performant Service	✔
Real-time C++ Simulation Loop	✔
Application Communication	✔
Continuous Integration	✔
Data-Driven Architecture	✔
Front-end + Local Backend	✔
Custom Tools Development	◆
2D Graphical Output	◆
3D (strong students only)	✘
Networking System	●
RBAC & Audit Logging — roles + immutable action trail (new)	✔

How each requirement applies:

> *Platform fit:* **Web browser on mobile + Native PC (operator ops)** — public results viewing is web-first (WASM build); the operator/admin dashboard is the PC ops side.

Front end: Choropleth Singapore map (district polygons, animated color transitions), party comparison charts, live results ticker. **Languages:** C++, GLSL (via WASM); HTML/JS/CSS shell.

Back end: Election-feed simulator & ingest engine, tally/projection engine, audit log, Firebase persistence & fan-out. **Languages:** C++.
Backend scope: Server-indicated (T2) — many viewers sharing one live result state; Firebase bridges T1, a real push server is the T2 path.

- *Simulation loop:* the election-day engine — simulated vote batches arriving per district, ordered ingest, running tallies and projections recomputed continuously, map/charts animating in real time.
- *Data-driven:* districts (boundary polygons), parties, candidates and feed scenarios all defined as data; swapping a scenario file replays a different election night.
- *Custom tool (adapted):* election setup editor — districts, parties, candidates and feed scenarios.
- *External DB:* result snapshots and viewer fan-out. *Auth:* operator/admin accounts (ties into the purple requirement). *Analytics »:* per-district viewership and query stats.
- *2D (adapted):* custom-rendered choropleth map with polygon tessellation, plus custom-rendered comparison charts. *3D:* not needed. *Networking:* ● live push to web viewers (WebSockets; Firebase bridge in T1).
- *RBAC & audit logging:* operators vs. admins vs. public viewers; every ingested batch, correction and result amendment lands in an immutable audit trail — election-integrity by construction.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the GovTech pool in `milestone-goals-pool-per-category.md` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**
  Tech & dependencies
  - **Tech:** SDL2 window (PC) and Emscripten/WASM shell (web) over one C++ core; fixed-timestep loop driving feed ingest, tally updates and map animation; C++17/20, CMake, Git.
  - **Prerequisites:** None — foundation of this project.
  - **Feeds into:** Every other goal; M1 G4 ingest and M1 G14 simulator tick inside this loop.
- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**
  Tech & dependencies
  - **Tech:** Districts (boundary polygons), parties, candidates and feed scenarios as JSON (nlohmann/json) with hot-reload; GeoJSON import for real Singapore boundaries.
  - **Prerequisites:** G1 (loop & hooks).
  - **Feeds into:** M1 G5 (map geometry), M1 G14 (scenario definitions), M2 G2 (editor edits this data).
- **G3 ★ Debugging, Profiling, Stress Test & CI**
  Tech & dependencies
  - **Tech:** ImGui overlays (batches/s, tally latency, render ms), Tracy profiling, Catch2 stress tests replaying full election nights at high speed, GitHub Actions Windows+Linux matrix plus a WASM build job.
  - **Prerequisites:** G1 (loop to instrument).
  - **Feeds into:** M2 G3; deterministic replays underpin M2 G10 projection tests.
- **G4 Real-time Feed / Queue Engine** — ordered ingest of per-district vote batches; the pipeline everything downstream consumes.

Tech & dependencies

- **Tech:** Ordered batch queue with timestamps, idempotent ingest (corrections re-apply cleanly), running per-district and national tallies in C++.
- **Prerequisites:** G1 (loop), G2 (district/party model).
- **Feeds into:** M2 G10 (projections read the tallies), M2 G7 (every batch is audited), the live map coloring.

- **G5 2D Map / Dashboard Rendering** — the custom-rendered Singapore choropleth and live charts; the face of the product.
  Tech & dependencies
  - **Tech:** Polygon tessellation of district boundaries to OpenGL triangles, choropleth coloring with animated transitions, pan/zoom camera, FreeType labels, custom bar/line charts.
  - **Prerequisites:** G1 (render loop), G2 (boundary data).
  - **Feeds into:** M2 G11 (report visuals), M2 G2 (editor live preview); every visible pixel of the demo.
- **G14 Domain Simulation Core** — the election-day feed simulator (adjustment 1): seeded scenarios that make the system testable and demoable.
  Tech & dependencies
  - **Tech:** Seeded scenario playback — turnout curves, batch arrival schedules, per-district party distributions, correction events — generating the vote stream M1 G4 ingests.
  - **Prerequisites:** G1 (clock), G2 (scenario data).
  - **Feeds into:** All development and demos; M2 G10 validates projections against simulated ground truth.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**
  Tech & dependencies
  - **Tech:** Tracy on ingest/tessellation paths, ASan on MSVC, batching for map redraws; validation of the WASM build on mobile browsers at full feed rate.
  - **Prerequisites:** All M1 goals.
  - **Feeds into:** Smooth election-night demo on phones; headroom for M2 G10.

- **G2 ★ Content Editor — Core** — the election setup editor: districts, parties, candidates, feed scenarios.
  Tech & dependencies
  - **Tech:** Editor for district metadata, party/candidate lists and feed-scenario parameters with JSON round-trip and an undo/redo command stack.
  - **Libraries/Software:** Dear ImGui, nlohmann/json, project's OpenGL map renderer (live preview)
  - **Prerequisites:** M1 G2 (data model), M1 G5 (map view), M1 G14 (scenario schema).
  - **Feeds into:** Every configured election; scenario variety for the showcase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
  Tech & dependencies
  - **Tech:** CMake presets, clang-format + clang-tidy gates, GitHub Actions matrix producing the Windows build and the deployable WASM bundle, replay-based regression tests.
  - **Prerequisites:** M1 G3 (incl. WASM job).
  - **Feeds into:** The shareable web demo URL; guards tally determinism.
- **G7 RBAC & Audit Logging** — the category's purple requirement: roles plus an immutable trail of every batch and correction.
  Tech & dependencies
  - **Tech:** Role model (admin / operator / public viewer) enforced across editor and ingest, append-only audit log of batches, corrections and amendments with actor + timestamp, audit viewer UI.
  - **Prerequisites:** M1 G4 (ingest events), Firebase Auth basics.
  - **Feeds into:** The election-integrity story of the demo — show a correction, then show its audit trail.
- **G10 Prediction Engine** — winner projections and swing analysis computed in C++; the "broadcast studio" feature.
  Tech & dependencies
  - **Tech:** Running projections per district (share of count reported, historical baselines, confidence bands), swing vs. previous election, "projected winner" calls with thresholds.
  - **Prerequisites:** M1 G4 (tallies), M1 G14 (ground truth to validate against), M2 G1 (headroom).

    - **Feeds into:** M2 G11 reports; the most impressive live moment of the showcase.
- **G11 Reports & Transparency Dashboards** — final results, party comparisons and exportable summaries.
  Tech & dependencies
    - **Tech:** Post-election dashboards — party comparison, turnout, seat totals, per-district breakdowns — custom-rendered and exportable (PNG/CSV) for transparency.
    - **Prerequisites:** M1 G4 + M2 G10 (data and projections), M1 G5 (chart rendering).
    - **Feeds into:** The final deliverable a viewer or journalist would actually use.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 0 Art — a data-viz/iconography surface (map styling, party palettes) fits GovTech's 0-artist norm; with 0 artists the Design conditionals are mandatory picks, so the second designer slot is strongly advised (and used below).

Track goals — suggested Design/Art picks and TEAM notes for this project; deliverables & quality ladders live in the Design, Art and TEAM pool documents.

- **Design M1:**
  - D1 ★ Features DD — viewer + ops blueprint (who watches, who operates the feed), trust & neutrality mandate
  - D2 Prototype — map → constituency drill-down → live update loop, both roles clickable
  - D5 Status & alert conventions spec — live-update ergonomics (counting/called/confirmed states, change flashes)
  - D11 Art POC (conditional — no artist) — civic map styling, colorblind-safe party palette, iconography
- **Design M2:**
  - D1 ★ Features Design Iteration & Showcase Readiness
  - D6 Service polished — feed → tally → map update → viewer, end to end on an election-night scenario
  - D8 Status & notification pass — result-confirmed moments with clarity and neutrality
  - D11 Art Implementation Quality (conditional)
- **UI/UX M1:**
  - U1 ★ UI Standards & Wireframe Baseline — the D5 conventions and wireframes are assessed in this review (one flow map serves both)
- **UI/UX M2:**
  - U2 ★ UX Validation & Final Polished UI — UX validation + final polished UI
  - U1 ★ Functional UI & Usability — GovTech screens: viewer map home · constituency drill-down · party comparison · ops/feed dashboard · settings · help; both roles navigable unaided

- **Art:** 0 artists — the Design conditionals (M1 D11 Art POC / M2 D11 Art Implementation Quality) cover the art surface.
- **TEAM M1:**
  - TA1 ★ Audio Direction & Plan — restrained civic cue set (result update, constituency called, alert)
- **TEAM M2:**
  - TA1 ★ Audio Integration & Quality — GovTech floor: no BGM; ≥5 cues (result update, constituency called, alert, error, success), every cue paired with a visual equivalent
  - TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase, prototype demo, team post-mortem (wk14)

Generated by the Project Proposal Assessor skill · grounding: live folder · references verified 2026-07-05 · track goals & unified workbooks retrofitted 2026-07-12.